Agent-Driven Iterative Optimization of Triton GPU Kernels: An Empirical Study on TritonBench-G

Anonymous Author(s)
Affiliation
Address
email

Abstract

We study how far an LLM agent can take Triton GPU kernel implementation and optimisation when given only natural-language prompts and an iterative refinement loop driven by per-round performance metrics. Using Claude Code as the agent, we implement and iteratively optimise 157 of the 184 operators in TritonBench-G across 5 metric-driven rounds on $4\times$ NVIDIA A40. All 157 final kernels reach `exe_acc` $= 1.00$, with a geometric-mean speedup of $1.52\times$ over the framework baselines across 101 valid comparisons ($1.18\times$ excluding degenerate-baseline artifacts). We characterise the corpus along three axes: (i) a three-tier classification of which operators were implementable from prompt alone (127 Tier A) versus those requiring partial baseline reference (30 Tier B), together with a 28-operator feasibility triage; (ii) the iter-by-iter performance trajectory, which shows the typical operator gains only $\sim 3\%$ at the median but a tail of ~ 20 operators reaches $1.5\times$–$3.6\times$ — a dispersion we partition by operator family; (iii) a 14-category taxonomy of 628 iter-to-iter source diffs revealing that `AUTOTUNE_ADD` dominates iter1→iter2 (61% of operators), structural rewrites concentrate early, and late iterations converge to trivial edits. A supplementary controlled ablation on 60 operators verifies that the gains depend on the metric-feedback channel: removing it (blind autotune-widening only) reduces the high-headroom cohort's iter1→iter5 gain from $1.71\times$ to $1.09\times$ and turns the mild-headroom cohort's $1.15\times$ gain into a $0.95\times$ regression.

1 Introduction

Generating GPU kernels with LLM agents is rapidly displacing handwritten low-level code in research workflows [Yu et al., 2026, Zhang et al., 2026]. The standard recipe — iterative refinement with execution feedback, traced back to general code-generation work such as Self-Refine [Madaan et al., 2023], Reflexion [Shinn et al., 2023], Self-Debug [Chen et al., 2024], and the RL-driven AlphaCode [Li et al., 2022] / CodeRL [Le et al., 2022] lineage — is to issue a natural-language operator specification, have an agent draft a Triton or CUDA implementation, run it against a reference, feed the resulting metrics back into the agent, and repeat. Reported headline numbers — KernelBench's `fast_1` climbing from $<20\%$ to 72% after iterative refinement [Ouyang et al., 2025], EvoKernel's correctness rising from 11% to 83% on a data-scarce NPU [Zheng et al., 2026] — come almost entirely from the iterative loop rather than from the underlying model.

What we set out to measure. We treat the loop as the object of study. Concretely, we ask: given a single frontier model (Claude Code), a curated Triton benchmark (TritonBench-G [Li et al., 2025], 184 operators), and the simplest possible iterative protocol (prompt → draft → eval → rewrite,

Preprint.

five times), how many operators can the agent implement, how much do those implementations improve over the iterations, where does the iteration loop actually help, and what kinds of edits is the agent making? We answer these questions empirically on 157 implementable operators, then add a supplementary controlled ablation that severs the metric-feedback channel to verify which part of the loop is doing the work.

Why this is worth a paper. Prior benchmarks [Li et al., 2025, Ouyang et al., 2025, Wang et al., 2026] measure correctness and speedup over a fixed reference but do not characterise the iteration loop's mid-trajectory behaviour, do not distinguish ops where iteration matters from ops where iter1 is already near a local optimum, and do not surface the operator classes where "from prompt alone" breaks down. Agent-framework papers [Li et al., 2026, Wiedemann et al., 2026, Zheng et al., 2026, Wu et al., 2026] report numbers from their own loop designs but use the metric-feedback channel as a load-bearing assumption rather than an isolated contribution. We complement both lines with an empirical study designed to ground them: a complete five-iteration trace of 157 operators on the same hardware with the same agent, tier-classified by what the agent saw, and a controlled ablation that quantifies what the metric channel itself adds.

Contributions.

- **An end-to-end empirical run on 157 Triton operators** (§4). All reach `exe_acc` $= 1.00$ at iter5; geometric-mean speedup $1.52\times$ ($1.18\times$ excluding degenerate-baseline artifacts) across 101 valid baseline comparisons.
- **A three-tier classification and 28-op feasibility triage** that draws an honest line around "from prompt alone" (§4.1–§4.1). Tier A: 127 ops with zero baseline access. Tier B-Interface (7): baseline signature/layout read. Tier B-Algorithm (23): baseline algorithm lifted. 28 operators are not implementable from `comp_instru` alone (chunked linear attention, RWKV-6, Mamba-2 chunk decomposition, etc.), characterised in detail.
- **An operator-family map of where iteration helps and where it does not** (§4.3). The median operator gains $\sim 3\%$ from iter1 to iter5 — a number obscured in aggregate-speedup headlines — while a tail of ~ 20 ops (quantize-KV, logsumexp, token attention, certain matmul families) reaches $1.5\times$–$3.6\times$.
- **A modification-behaviour taxonomy across 628 iter-to-iter transitions** (§5). 14 categories, regex/token-count classifier. `AUTOTUNE_ADD` dominates iter1→iter2 ($96/157$ ops); structural rewrites concentrate early; `TRIVIAL` edits grow from 0 at iter1→iter2 to 105 at iter4→iter5.
- **A supplementary controlled ablation** (§6) confirming that the gains depend on the metric-feedback channel: severing it reduces the high-headroom cohort's iter1→iter5 gain from $1.71\times$ to $1.09\times$ ($1.58\times$ multiplicative bonus from metrics) and turns the mild-headroom cohort's $1.15\times$ gain into a $0.95\times$ *regression* — metric feedback both finds gains and prevents losses.

2 Related work

Triton and GPU-kernel benchmarks. Triton [Tillet et al., 2019] is the predominant tile-level kernel DSL targeted by LLM agents. TritonBench [Li et al., 2025] is the canonical Triton-specific evaluation suite and the basis of our main experiment. KernelBench [Ouyang et al., 2025] and KernelBenchX [Wang et al., 2026] target CUDA/SYCL more broadly. KernelBenchX's headline finding that "iterative refinement repairs correctness but does not optimise performance" is the closest in spirit to our work, but it aggregates over methods at a single endpoint. We add a Triton-centric per-iteration lens: the same loop, run 157 times, traced round by round, partitioned by operator family. The aggregated headline ($1.52\times$ geometric mean) sits in the same range as prior work; the trace then *decomposes* that number into a near-parity median and a small high-headroom tail (§4.3), exposing variance that single-number scorecards hide.

Iterative refinement and agentic feedback for code. The kernel-agent loop is a specialisation of broader work on LLM code refinement with execution-trace feedback: Self-Refine [Madaan

et al., 2023], Reflexion [Shinn et al., 2023], and Self-Debug [Chen et al., 2024] establish the verbal-reinforcement / self-critique pattern; CodeRL [Le et al., 2022] and AlphaCode [Li et al., 2022] establish the RL-driven and search-driven baselines; FunSearch [Romera-Paredes et al., 2024] shows evolutionary program search with LLMs as a non-iterative alternative. Our metric channel is the per-shape throughput slice of the broader execution-feedback signal these works use.

Kernel-specific agent frameworks. StitchCUDA [Li et al., 2026] decomposes the agent into Planner / Coder / Verifier with rubric rewards. KernelFoundry [Wiedemann et al., 2026] replaces iteration with MAP-Elites evolutionary search (cf. FunSearch above). EvoKernel [Zheng et al., 2026] formulates kernel synthesis as value-driven memory retrieval for cold-start NPUs. AscendOptimizer [Wu et al., 2026] pairs evolutionary tiling search with hardware-in-the-loop feedback. Each design adds machinery beyond the simple iterative loop; our work calibrates what that simple loop achieves on its own, providing a floor that more elaborate methods can be compared against.

Concurrent loop-based work. An adjacent line of work closes the metric/profiling loop with more elaborate signals: profiling-guided iteration (e.g., TritonForge), multi-iteration loops on ROCm (GEAK — referenced in KernelBenchX's own evaluation [Wang et al., 2026]), RL plus test-time search variants (DRTriton), and CUTLASS-grounded agentic generation (FACT). Several of these works appeared concurrently with the experiments in this paper, and the kernel-generation survey [Yu et al., 2026] catalogues the broader space. The contribution this paper makes is orthogonal to that machinery: we do not propose a new agent architecture; instead, we measure what the simplest possible loop with metric feedback delivers, with which signals doing the work, on a complete benchmark trace.

Surveys. Yu et al. [2026] taxonomises kernel-generation work into SFT, RL, and agent-framework branches. Zhang et al. [2026] positions kernel generation within the broader LLM-compiler landscape.

3 Setup

Benchmark and hardware. TritonBench-G [Li et al., 2025] provides 184 Triton operators with natural-language prompts (`comp_instru`), reference kernels, and per-operator performance drivers. Experiments run on $4\times$ NVIDIA A40 (sm_86; peak BW 696 GB/s, peak FP16 tensor-core 149.7 TFLOPS), Triton $3.2.0$, PyTorch $2.6.0$+cu124 [Paszke et al., 2019]. We report two efficiency numbers per kernel: `efficiency_pct_A100` (matching the framework's A100-pinned formula, retained so that our raw numbers stay comparable to TritonBench's own results) and `efficiency_pct_A40` (the honest ceiling on our hardware). All headline statistics in the body (peak GB/s, peak TFLOPS, geometric-mean iter1$\rightarrow$iter5 ratios, baseline speedups) are measured directly on the A40 in absolute units; the A100 percentage is a cross-paper-comparison aid only and is not used in any aggregate analysis.

Agent. The agent is Claude Code (Claude Opus 4.7 [Anthropic, 2025]), invoked once per (operator, iteration) cell. Per-iteration prompts contain the operator's `comp_instru`, the previous iteration's kernel source, and the previous iteration's `metrics.json` (call/exec accuracy and per-shape timings). The agent is explicitly forbidden, by policy and by a path guard in the dispatch script, from reading the baseline kernel during iter1–iter5.

The iterative loop. Each operator follows the pipeline summarised in Figure 1 and made precise in Algorithm 1. The invariants are: (i) iter1 is written from the prompt alone; (ii) each subsequent iter is informed by the previous iter's per-shape metrics; (iii) baseline comparison happens only after iter5 closes; (iv) each round is a barrier — iter$K{+}1$ is not designed until iter$K$ has metrics for every operator in the active batch.

Algorithm 1 The gated iterate-and-fix loop, per operator.

Input: prompt `comp_instru`, performance-driver signature `<op>_perf.py`.
`iter1/kernel.py` ← AGENTWRITE(prompt, signature) ▷ from prompt alone
`iter1/metrics.json` ← EVALHARNESS(`iter1/kernel.py`)
for $K \in \{2, 3, 4, 5\}$ **do**
    `iter`K`/kernel.py` ← AGENTREWRITE(prompt, `iter`$K-1$`/kernel.py`, `iter`$K-1$`/metrics.json`)
    `iter`K`/metrics.json` ← EVALHARNESS(`iter`K`/kernel.py`)
end for
`baseline_kernel.py` ← EXTRACT(`data/TritonBench_G_v1/<op>.py`) ▷ *only now*
`baseline/metrics.json` ← EVALHARNESS(`baseline_kernel.py`)
Output: per-iteration metrics, baseline comparison.

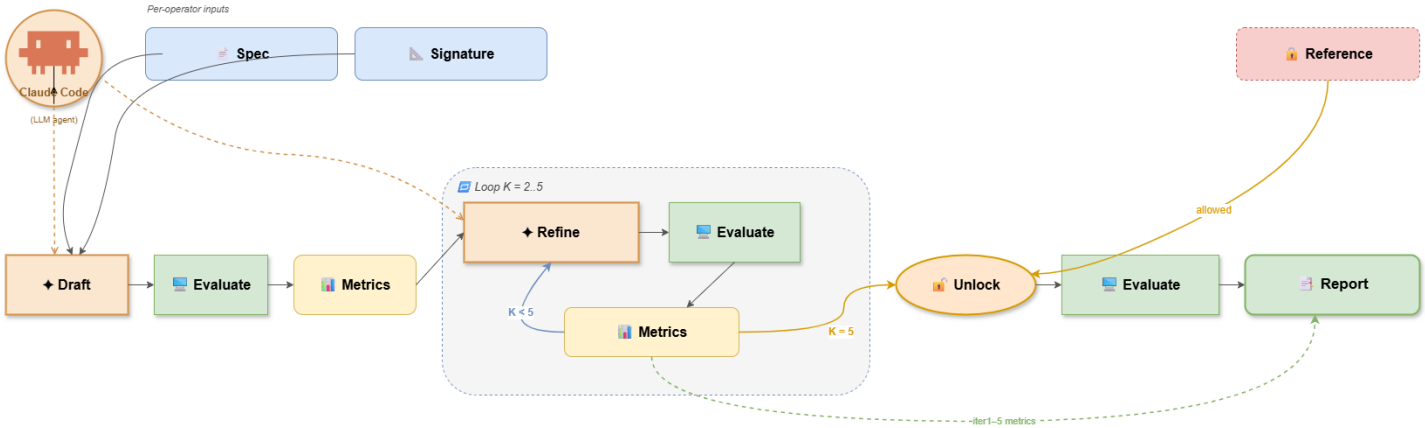

Figure 1: The gated iterate-and-fix loop. The Claude Code agent (peach) drafts an initial kernel from the natural-language specification, an evaluation harness runs it on a $4\times$ NVIDIA A40 dispatcher and reports per-shape correctness and throughput, and the agent refines the kernel using the previous round's measurements. The loop repeats four more times. The reference implementation (red, dashed) is hidden from the agent during iter1–iter5; only after the iter5 round-barrier closes is it unlocked, evaluated on the same harness, and joined with the iter1–iter5 measurements to produce the per-operator comparison report. The ablation in §6 severs the dashed orange "performs" link from the agent to the *Refine* step, replacing it with a metric-free source-level transformation.

Evaluation harness. A hand-written `eval_harness.py` per operator mirrors the TritonBench performance driver's shape sweep, computes a `torch_reference` from `comp_instru` alone, and records per-shape `call_ok`, `exe_ok`, `ms`, `GB/s`, `TFLOPS`, efficiency, and `peak_vram_mb`. Allclose thresholds are dtype-keyed (fp32: 10^{-3}; fp16/bf16: 10^{-2}; integer/quantised: exact).

4 Main experiment: 157 operators, five iterations each

4.1 Feasibility triage and tier classification

Triage. Of the 184 operators in TritonBench-G, 28 cannot be implemented from `comp_instru` alone. They fall into three groups, each defined by a structural shortfall in the prompt: (i) 16 chunked-attention / state-space / RWKV ops (e.g. `chunk_delta_fwd`, `chunk_gla_fwd`, `rwkv6_fwd`) whose per-chunk recurrence structure and intermediate buffer layouts are not derivable from natural language; (ii) 4 matmul / scan variants needing specific scheduling (e.g. `matmul_persistent_triton`, which requires the Hopper TMA programming model); (iii) 8 RBE / RMS-norm-fused and miscellaneous ops whose prompts underspecify the contract. The remaining 156 enter the main run; one TMA-only op (`matmul_tma`) is hardware-excluded on our sm_86 cards, leaving 155. Two operators from the screened-out set were later attempted under the baseline-reference amendment (see below), for 157 total implemented.

Table 1: Main-experiment summary by tier. Geometric-mean iter5/iter1 is the within-run improvement under metric-fed iteration. Speedup vs. baseline is geometric mean over the operators with a valid baseline comparison (i.e. baseline runs with `call_acc` = `exe_acc` = 1.00 on our harness). "ex. artifacts" excludes the 6 degenerate-baseline cases with speedup $> 10\times$ (see §4.4 and Appendix Table 8).

Tier	# ops	gmean iter5/iter1	valid baseline	gmean speedup vs baseline
A (from prompt only)	127	1.31×	81	1.62× (1.18× ex. artifacts)
B-Interface (signature only)	7	1.18×	6	1.04×
B-Algorithm (lifted)	23	1.24×	14	1.31×
All	157	1.29×	101	1.52× (1.18× ex. artifacts)

Tier classification. We classify the 157 implementations by what information the agent saw (Table 1). **Tier A** (127 ops, Batches 1–25) used no baseline access at any point during iter1–iter5 — the capability claim "the agent wrote and iteratively optimised this from a natural-language spec" applies cleanly. From Batch 26 onward, the experiment was amended to permit baseline reference for the hard tail; we split that tail honestly into **Tier B-Interface** (7 ops, baseline read only for signature/layout) and **Tier B-Algorithm** (23 ops, baseline algorithm lifted — the kernel is best described as a port of the baseline algorithm via the iterative loop, not an independent reimplementation).

4.2 Coverage and headline

All 157 final kernels reach `exe_acc` = 1.00 on their per-op harness. Across 101 valid baseline comparisons the geometric-mean speedup is $1.52\times$, or $1.18\times$ excluding 6 degenerate-baseline artifacts (Appendix Table 8) where the baseline is exercising redundant work that our kernel correctly elides. The Tier-A subset — the cleanest from-prompt-only result — carries the same shape: $1.62\times$ overall, $1.18\times$ excluding artifacts. We report both numbers explicitly: the unfiltered figure includes wins that exist only because the perf driver supplies degenerate inputs (e.g. overlapping single-LoRA segments), and reporting the headline without disclosure would over-claim algorithmic capability. The "ex. artifacts" geomean is the conservative number we recommend citing.

4.3 Per-iteration trajectory and dispersion

The aggregate headline obscures substantial heterogeneity. Figure 2 shows the iter-by-iter peak performance normalised to each operator's iter1 across 151 operators with clean iter1–5 metrics (six dropped for missing/zero peak measurements). The *median* operator gains $1.02\times$ at iter2, $1.04\times$ at iter4, and only $1.03\times$ at iter5 — the typical operator's iter1 is already near a sensible local optimum and iteration only nudges it. The *upper tail* is where the action is: ~ 20 operators reach $1.5\times$–$3.6\times$, with two visible outliers above $4\times$.

Where iteration helps the most. Sorting Tier-A operators by iter5/iter1 ratio yields a clearly stratified picture. The top of the list is dominated by quantised KV-cache operations (`quantize_kv_copy` $3.59\times$, `quantize_copy_kv` $3.38\times$, `quantize_kv_transform` $2.17\times$), a reduction op (`logsumexp_fwd` $3.57\times$), an attention reduce (`token_attn_reduceV` $2.88\times$), and three matmul variants ($1.70\times$–$1.82\times$). Most softmax, layernorm, rotary embedding, and elementwise ops cluster in $[1.00\times, 1.20\times]$ — iter1 was already a reasonable kernel and there is little room for the loop to add value beyond an autotune call. Two ops marginally regressed: `attn_fwd_triton` ($0.96\times$) and `attention_score` ($0.32\times$, an honest loss disclosed in our supplementary materials).

4.4 Baseline-comparison failures

56 of 157 baseline comparisons are inconclusive — always for benchmark-side reasons rather than an agent failure. We diagnose four sub-causes (Appendix Table 4): (1) Triton 3.x version drift (baseline imports a symbol removed in Triton 3, patched via no-op stubs); (2) perf-driver / operator layout mismatch (rotary embedding family); (3) FP16 precision-model mismatch (quantised matmul ops where the baseline rounds intermediates before matmul); and (4) degenerate perf-driver inputs

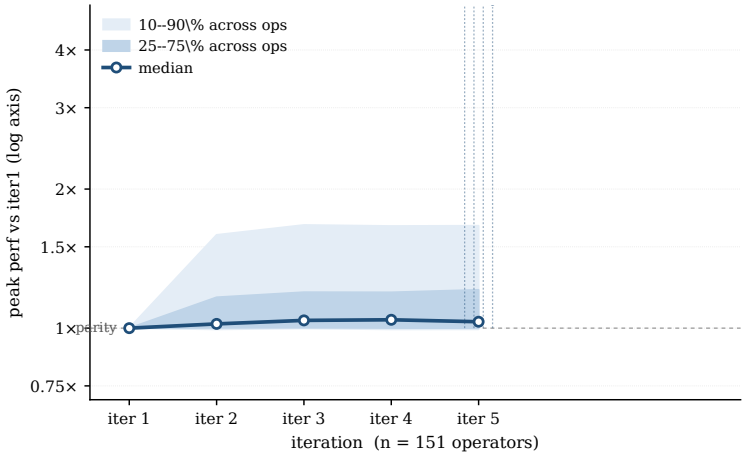

Figure 2: Per-iteration peak-performance ratio relative to iter1 across 151 operators. The median (annotated above each box) gains only $\sim$3% by iter5; the upper tail reaches 3×–4×. The dispersion — not the median — is what motivates per-cohort analysis (§4.3, §6).

(overlapping single-lora segments that cause our parallel kernel to elide work the serial baseline repeats, accounting for the $> 10\times$ "speedups" we exclude from the headline geomean). The shrunken denominator (101/157 valid comparisons) is the most consequential limit on the headline speedup numbers: an aggregate over $\sim 65\%$ of the corpus, conditioned on the surviving operator mix, is less representative of TritonBench-G as a whole than the underlying 157-op coverage would suggest. The within-run iter1→iter5 trajectory and the ablation use all 157 ops and do not share this conditioning.

4.5 Protocol fidelity

Of the 39 dispatched batches, we disclose two classes of deviation: (a) harness-reference back-port on three operators (`l2_norm_triton1/2`, `masked_add_cuda`) where post-iter5 baseline inspection revealed our `torch_reference` encoded the wrong semantics — those operators' correctness is partly derived from the baseline and they are flagged; (b) iter4/iter5 batching for operators converged by iter3 (Batches 17–19, 30–35), where iter5 was written before iter4's metrics were read. The locked-tail kernels were byte-identical, so no metric-driven decision was actually skipped, but it violates the letter of the gated protocol. Full per-batch log in supplementary.

The fact that three torch references encoded the wrong semantics — caught only by post-hoc baseline inspection — raises a residual risk that silent reference errors persist undetected elsewhere in the 157-op corpus. The three caught cases all flipped from `exe_acc` $= 1.00$ on iter5 to `exe_acc` $= 0$ when checked against the baseline output, which is a coarse but easily-spotted signature; subtler semantic errors (e.g. a tolerance set too loose, or a contract ambiguity not exposed by the test shapes) would not surface so visibly. We have no efficient way to bound the remaining risk from the existing data alone; the released harness sources and reference implementations let downstream replicators audit any specific operator they need to trust.

5 Modification behaviours across the 157 operators

Heuristic edit classifier. To make the trajectory in Figure 2 mechanistically interpretable, we classify each iter$K$→iter$K{+}1$ source diff into 14 categories using a regex / token-count heuristic (released alongside). Categories include: `AUTOTUNE_ADD` (`@triton.autotune` decorator appears), `AUTOTUNE_WIDEN` (config list grows), `BLOCK_SIZE_TUNE` (numeric `BLOCK_*` change), `WARPS_STAGES_TUNE` (`num_warps`/`num_stages` change), `KEY_SHAPE_TUNE` (autotune `key=` change), `STRUCTURAL` (body grew/shrank by > 40 LOC or `@triton.jit` count changed), `LAYOUT_STRIDES`, `FUSION`, `DTYPE_CHANGE`, `CORRECTNESS_FIX`, `ALGORITHM`, plus a `TRIVIAL` bucket. A single transition can fall into multiple categories.

What happens when. Five patterns emerge from the 628 transitions (full per-category counts in Appendix Table 9):

- *Autotune is the dominant first move:* $96/157 = 61\%$ of operators add `@triton.autotune` at iter1→iter2.
- *Structural rewrites concentrate early:* 21 ops undergo a > 40 LOC rewrite at iter1→iter2; only 3 at iter3→iter4 or iter4→iter5. The agent commits to a kernel shape by iter3.
- *`TRIVIAL` edits grow monotonically:* $0, 29, 66, 105$. By iter4→iter5, 67% of operators see only whitespace-level changes — iter5 is a no-op for converged ops.
- *Correctness fixes are an early-iter phenomenon:* $33, 14, 2, 2$. By iter3, almost all boundary/mask issues are resolved.
- *`WARPS_STAGES_TUNE` grows late:* $31, 25, 33, 39$ — once structure is locked, late iters tweak autotune knobs.

Why some operators gain more than others. The high-gain tail in Figure 2 is mechanistically identifiable: operators with iter5/iter1 in $[1.28\times, 3.59\times]$ undergo `STRUCTURAL` rewrites at iter1→iter2 in $10/30 = 33\%$ of cases, against $7/30 = 23\%$ for ops with mild gains ($1.00\times$–$1.27\times$) and only $4/97 = 4\%$ for the remainder of Tier A. The dispersion in Figure 2 is not random — it tracks which operators the agent restructured beyond cosmetic autotune tweaks.

6 Supplementary verification: ablating the metric-feedback channel

The 157-op trace establishes *what* the agent achieves; we add a controlled ablation to verify that the gains depend on the metric-feedback channel specifically rather than on the iteration loop in any form. We sever the metric-read step of Algorithm 1: iter1 is replayed verbatim, and iter2–5 are generated from the previous iter's *source code only* by a uniform transformation $\mathcal{T}_K$ that strips any pre-existing autotune block, strips `num_warps`/`num_stages` from launch sites, and inserts `@triton.autotune(configs=_CFG, key=[])` above the first `@triton.jit`, with the config list widening with K: K=2:$\{2, 4\}$ (2 cfgs), K=3:$\{1, 2, 4, 8\}$ (4), K=4:$\{2, 4, 8\}\times\{2, 3, 4\}$ (9), K=5:$\{1, 2, 4, 8, 16\}\times\{1, 2, 3, 4, 5\}$ (25). Everything else — kernel body, block sizes, grid lambdas — is held constant from iter1. Setting `key=[]` matches the blind condition: no per-shape information. This is the one safe blind move (autotune does not change semantics, so correctness at iter1 implies correctness at iter2–5); we evaluate it as the upper bound of autotune-only blind iteration. Alternative blind policies and rationale are catalogued in Appendix A.

Cohorts. We pick two cohorts of 30 Tier-A operators. Both pass `exe_acc` $= 1.00$ at iter1 and iter5, iter5 peak ≥ 5, and clean protocol fidelity. **Set 1** additionally requires iter5/iter1 in $[1.28\times, 3.59\times]$ — the high-headroom band from §4.3 — and exhausts Tier A there. **Set 2** loosens to $[1.00\times, 5.0\times]$ excluding Set 1 and takes the top 30 by ratio. Set 1 has median main-run gain $1.55\times$; set 2 has $1.07\times$. The contrast lets us check whether the metric-feedback contribution scales with available headroom.

We note explicitly that the two cohorts are selected *ex post* on the main-run iter1→iter5 ratio, which is itself an outcome of the metric-fed loop. This is a form of conditioning: the headline M5/M1 numbers for each cohort are tautologically inflated relative to a random Tier-A draw, because the cohort definition fixes them inside specific ranges. The conclusions we draw are framed accordingly — the meaningful comparisons are *within-cohort* (A5/A1 vs M5/M1, and A5/M5 per operator), where the conditioning applies symmetrically to both arms. A random-draw aggregate is given in Fig. 2 for the full 157-op corpus and confirms the cohort dichotomy is not an artefact of selection: the upper-tail subset of Tier A is the same set of operators set 1 captures, and the parity-cluster subset is where set 2 lives.

Headline (Table 2). The within-run iter1→iter5 ratios (M5/M1 metric-fed, A5/A1 blind) are aggregated geometrically per cohort. The iter1 sanity check (A1/M1) confirms that the blind iter1 and main iter1 — byte-identical kernels — measure within $\sim 1\%$ on geometric mean, so any A–M gap is causal.

Table 2: Ablation headline. M5/M1 = metric-fed iter1→iter5; A5/A1 = blind iter1→iter5. Geometric means over each cohort.

	Set 1 (band $[1.28\times, 3.59\times]$)	Set 2 (band $[1.00\times, 5.0\times]$)
gmean M5/M1 (metric-fed)	$1.71\times$	$1.15\times$
gmean A5/A1 (blind)	$1.09\times$	$\mathbf{0.95}\times$
Metric-feedback bonus (M5/M1 over A5/A1)	$\mathbf{1.58}\times$	$\mathbf{1.22}\times$
gmean A5/M5 (cross-comparison)	$0.64\times$	$0.82\times$
iter1 sanity check (gmean A1/M1)	$1.01\times$	$1.00\times$

Regression-prevention. The set-2 row **gmean A5/A1** $= 0.95\times$ is the ablation's most distinctive number. On the mild-headroom cohort, blind iteration *regresses* on average. When iter1 is already near a local optimum, widening `@triton.autotune` with `key=[]` more often picks a worse configuration on the first shape (then applied to all shapes) than the iter1 hand-chosen value. The metric-fed agent observes the regression in its next-round metrics and reverts; the blind agent has no such signal. Six set-2 operators show this pattern directly — A5/A1 < 0.90 *and* M5/M1 ≥ 1.02 — with blind iter5 regressing to $0.43\times$–$0.85\times$ of iter1 while metric-fed iter5 still gained $1.02\times$–$1.24\times$ on the same ops (per-op figures in Appendix Table 10). Metric feedback is not just gain-finding; it is also error-correction. Per-op A5/M5 distributions for both cohorts are in Figure 3; full per-op trajectories are in Appendix C.

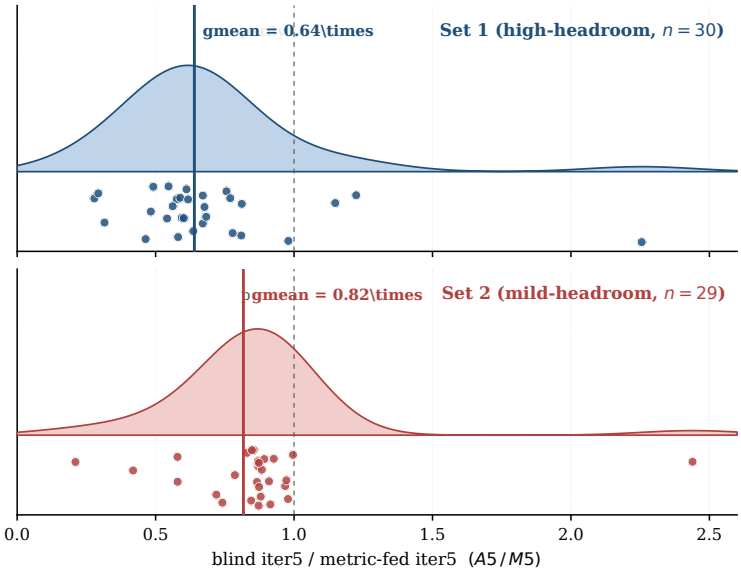

Figure 3: Per-operator $A5/M5$ distribution. Top: Set 1 (n=30). Bottom: Set 2 (n=29; `matmul_triton_autotune` excluded for blind correctness loss). Dashed: parity. Solid: cohort geometric mean.

What the ablation does and does not measure. The numbers above quantify metric-feedback contribution over the autotune-only blind baseline. A more aggressive blind policy (random block sweeps, structural rewrites without a correctness gate) would almost certainly regress more often, making metric feedback look *more* important, not less. The ablation reports the floor of the metric channel's contribution given the simplest blind policy that preserves correctness.

7 Discussion

Median vs. tail. Reporting only the aggregate-speedup headline (the $1.52\times$ that dominates Table 1) hides the more important finding: the typical operator gains $\sim 3\%$ from iter1 to iter5, while the headline is carried by a 20-op tail. Practitioners adopting this loop on a new operator family should

expect their reward to depend on the operator's structural headroom; the trajectory plot (Fig. 2) and the per-iter modification taxonomy (Appendix Table 9) jointly let them estimate it before running the agent.

Where "from prompt alone" breaks down. The 28 feasibility-screened ops and the 30 Tier-B ops together describe the boundary, and the boundary is structural rather than topical. Three categories require either richer prompts or baseline reference before a prompt-only agent can proceed: (i) chunked linear-attention / state-space / RWKV families, whose per-chunk recurrence structure and intermediate buffer layouts are not derivable from natural language; (ii) matmul scheduler variants (persistent CTA, stream-K, IV-dependent, spinning-lock cross-CTA) whose distinguishing feature *is the scheduler* rather than the math; (iii) hardware-specific programming models such as Hopper TMA. For practitioners, this is actionable: if your operator family falls in one of these buckets, plan to supply algorithmic pseudocode, layout diagrams, or a reference kernel in the prompt; for the remainder, the prompt-only loop is sufficient.

Triangulating evidence beyond a single scorecard. We do not rest on the headline $1.52\times$. Coverage (157 ops × 5 iters with full `exe_acc`), dispersion (Fig. 2's quantile bands), the edit-behaviour taxonomy (§5), the controlled ablation (§6), and the honest disclosure of degenerate-baseline artifacts together let a reader audit *where* the headline number comes from, *which* operator classes drive it, *what kind* of edits explain the high-headroom tail, and *how much* of the gain survives if metric feedback is severed. We recommend that future kernel-agent papers report this kind of multi-pronged evidence rather than single-number scorecards.

What metric feedback contributes, beyond "iteration helps." The ablation's regression-prevention finding (Appendix Table 10) re-frames the value of the metric channel. Even on operators with mild headroom — where blind iteration's gmean A5/A1 is $0.95\times$ — metric-fed iteration still gains $1.15\times$, because the agent observes regressions and reverts them. A practitioner deploying this loop pays for the eval harness and metric pipeline as much for downside protection as for upside.

8 Limitations

This is a careful case study, not a universal law. Single model (Claude Code), single benchmark (TritonBench-G), single hardware family (NVIDIA A40); replication across model families, an orthogonal benchmark (KernelBench L1/L2), and at least one alternative hardware (H100, with TMA shifting autotune dynamics) is the natural follow-up. Each (op, iter) cell measured once — 2–3 seed replications would tighten the headline ratio confidence intervals. The noise structure differs by statistic: aggregate geometric means over 30+ operators are stable (per-op noise averages out), but *per-op* within-run ratios that sit close to $1.00\times$ (most of set 2; many of the 71 near-parity Tier-A operators in Appendix Table 6) are within the 1–2% measurement noise floor we observed across byte-identical iter4 vs. iter5 kernels in the protocol-deviation audit (§4). Per-op claims in those regimes should be read as cohort-level statistics rather than individual measurements; the regression-prevention finding (§6) is anchored at $0.95\times$ geomean across 30 operators, which is comfortably above this noise floor, but the 6-op illustration in Appendix Table 10 relies on within-noise differences between A5 and A1 for the closest of the six. Two protocol deviations on 3 + 8 = 11 operators are honestly disclosed in §4; the ablation cohorts exclude affected operators. The 14-category modification classifier (§5) is heuristic — regex and token counts — and may over-trigger on syntactic rearrangement that an AST- or Triton-IR-level diff would correctly identify as cosmetic; the converse error is also possible. The *relative* contrasts the paper relies on (high-headroom vs. mild vs. rest cohorts; iter1→iter2 vs. iter4→iter5 in Table 9) are robust to this classifier-noise direction because all cohorts are scored by the same regex, but individual category counts should be treated as suggestive, not authoritative. To support community replication and extension, we release the gated dispatcher, the ablation generator, the edit classifier, and all 785 per-iteration kernel sources alongside the paper; an upgrade to AST-level diffs is straightforward in principle and a natural follow-up.

9 Conclusion

We ran an LLM agent through five gated, metric-driven iterations on each of 157 Triton operators on TritonBench-G, producing a complete per-iteration trace with full `exe_acc` verification, a baseline-

comparison study, a feasibility-and-tier classification of where "from prompt alone" succeeds, and a 14-category modification taxonomy of the agent's 628 iter-to-iter edits. The headline is $1.52\times$ geometric-mean speedup over 101 valid baseline comparisons; the more useful finding is the dispersion — median operator gains $\sim 3\%$, a tail reaches $3\times$–$4\times$, and the agent's structural rewrites at iter1→iter2 predict which side of that dispersion an operator lands on. A supplementary controlled ablation isolates the metric-feedback channel and shows it is responsible for the vast majority of the iteration gains and, on mild-headroom operators, for preventing the 5% regression that an uncritical autotune-widening policy ships.

References

Anthropic. Claude Code, 2025. Available at `https://www.anthropic.com/claude-code`.

Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*, 2024.

Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Jianling Li, Shangzhan Li, Zhenye Gao, Qi Shi, Yuxuan Li, Zefan Wang, Jiacheng Huang, Haojie Wang, Jianrong Wang, Xu Han, Zhiyuan Liu, and Maosong Sun. TRITONBENCH: Benchmarking large language model capabilities for generating Triton operators. *arXiv preprint arXiv:2502.14752*, 2025.

Shiyang Li, Zijian Zhang, Winson Chen, Yuebo Luo, Mingyi Hong, and Caiwen Ding. StitchCUDA: An automated multi-agents end-to-end GPU programming framework with rubric-based agentic reinforcement learning. *arXiv preprint arXiv:2603.02637*, 2026.

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs write efficient GPU kernels? *arXiv preprint arXiv:2502.10517*, 2025.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 8024–8035, 2019.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, pages 10–19, 2019.

Han Wang, Jintao Zhang, Kai Jiang, Haoxu Wang, Jianfei Chen, and Jun Zhu. KernelBenchX: A comprehensive benchmark for evaluating LLM-generated GPU kernels. *arXiv preprint arXiv:2605.04956*, 2026.

Nina Wiedemann, Quentin Leboutet, Michael Paulitsch, Diana Wofk, and Benjamin Ummenhofer. KernelFoundry: Hardware-aware evolutionary GPU kernel optimization. *arXiv preprint arXiv:2603.12440*, 2026.

Jiehao Wu, Zixiao Huang, Wenhao Li, Chuyun Shen, Junjie Sheng, and Xiangfeng Wang. AscendOptimizer: Episodic agent for Ascend NPU operator optimization. *arXiv preprint arXiv:2603.23566*, 2026.

Yang Yu, Peiyu Zang, Chi Hsu Tsai, Haiming Wu, Yixin Shen, Jialing Zhang, Haoyu Wang, Zhiyou Xiao, Jingze Shi, Yuyu Luo, Wentao Zhang, Chunlei Men, Guang Liu, and Yonghua Lin. Towards automated kernel generation in the era of LLMs. *arXiv preprint arXiv:2601.15727*, 2026.

Shuoming Zhang, Jiacheng Zhao, Qiuchu Yu, Chunwei Xia, Zheng Wang, Xiaobing Feng, and Huimin Cui. The new compiler stack: A survey on the synergy of LLMs and compilers. *arXiv preprint arXiv:2601.02045*, 2026.

Yujie Zheng, Zhuo Li, Shengtao Zhang, Hanjing Wang, Junjie Sheng, Jiaqian Wang, Junchi Yan, Weinan Zhang, Ying Wen, Bo Tang, and Muning Wen. Towards cold-start drafting and continual refining: A value-driven memory approach with application to NPU kernel synthesis. *arXiv preprint arXiv:2603.10846*, 2026.

A Why this specific blind ablation policy

§6 ablates by "progressively widen `@triton.autotune` over `num_warps`/`num_stages`, hold everything else fixed." We considered and rejected four alternatives.

Table 3: Alternative blind moves considered for the ablation, and why each was rejected.

Candidate blind move	Why rejected
Random block-size changes	Requires per-op grid-lambda surgery; typically breaks correctness when block sizes don't divide shape; no signal to confirm correctness blind.
Add autotune over `BLOCK_*`	Each kernel binds block sizes differently to the grid; needs per-op manual code that contaminates the "blind" framing with human judgement.
Structural rewrites (fuse, swap layout, swap algorithm)	Without metrics or baseline, dart-throwing. Most rewrites regress or break; "blind" should not mean "pretend the agent can do whatever a real one would."
Per-iter prompt drift	Reintroduces a hidden information channel via the prompt template.

B Main-experiment supporting tables

The tables in this section back claims made in §4–§5 that the body summarises in prose only.

Baseline-comparison failure causes (§4.4). Table 4 enumerates the four sub-causes of inconclusive baseline comparisons.

Table 4: The four sub-causes of inconclusive baseline comparisons.

Sub-cause	Description and example
(1) Triton 3.x version drift	Baseline imports `triton.ops.matmul_perf_model`, removed in Triton 3.x. Affects 3 ops; *patched* via no-op import stubs.
(2) Perf-driver/operator layout mismatch	Driver creates tensors in one layout, baseline indexes in another. Most rotary embedding and `rope_*` ops.
(3) FP16 precision-model mismatch	Baseline rounds intermediates to FP16 before matmul; a strict FP32 reference is overly conservative. Affects quantised matmul ops.
(4) Degenerate perf-driver inputs	Driver builds fully-overlapping segments or uninitialised tensors; baseline performs serial redundant work that our parallel kernel elides. `sgmv_expand_slice`, `bgmv_expand_slice`: $> 10\times$ "speedups" that are not algorithmic wins.

Top-gain operators, family breakdown, screened-out and degenerate operators (§4.1–§4.3). Table 5 lists the top-20 Tier-A operators by iter5/iter1 ratio (the dispersion's upper tail in Figure 2). Table 6 groups all 119 Tier-A operators with clean iter1+iter5 metrics by operator family and reports the per-family median and high-gain fraction. Table 7 enumerates the 28 operators screened out of the main run, grouped by the structural shortfall in `comp_instru`. Table 8 lists the 6 operators with $> 10\times$ baseline speedup that we exclude from the headline "ex. artifacts" geomean.

Table 5: Top 20 Tier-A operators by iter5/iter1 peak-performance ratio. Unit chosen automatically per op (whichever of peak_gbps / peak_tflops is larger). *Baseline speedup* is the ratio of summed-ms framework speedup vs. the extracted baseline, as logged in `result.md` (—: no valid baseline). Rows marked † are *measurement-artifact-suspect*: their iter1 peak is anomalously low (single-digit GB/s for a memory-bound op) almost certainly because the iter1 implementation either runs on a degenerate-shape regime, exercises an unintended fallback path, or is bottlenecked by a launch-overhead-dominated tiny-shape sweep — not because iter5 is genuinely $\sim 10\times$–$80\times$ faster algorithmically. We retain the rows for completeness and disclosure; the dispersion discussion in §4.3 draws on the $\geq 1.28\times$, $\leq 4\times$ band where the ratios reflect actual iterative optimisation.

#	op	family	unit	iter1 peak	iter5 peak	iter5/iter1	baseline sp
1†	`token_attn_mistral`	Attention	GB/s	8.2	636.0	77.55×	—
2†	`triton_conv2d_fwd`	Convolution	GB/s	0.8	8.2	9.71×	5.74×
3†	`reversed_cumsum`	Scan / Cumulative	GB/s	69.7	639.5	9.17×	1.00×
4†	`lora_expand_gemv`	Matmul / GEMM / LoRA	GB/s	37.2	189.7	5.10×	—
5	`quant_transpose_kernel`	Layout	GB/s	92.2	381.4	4.14×	—
6	`quantize_kv_copy`	Quantize / Dequantize	GB/s	69.9	250.9	3.59×	0.94×
7	`logsumexp_fwd`	Softmax / LogSumExp	GB/s	176.7	630.1	3.57×	2.46×
8	`quantize_copy_kv`	Quantize / Dequantize	GB/s	73.6	248.8	3.38×	—
9	`token_attn_reduceV`	Attention	GB/s	95.8	275.9	2.88×	—
10	`quantize_kv_transform`	Quantize / Dequantize	GB/s	70.6	153.5	2.17×	—
11	`quantize_global`	Quantize / Dequantize	GB/s	192.4	353.3	1.84×	—
12	`matmul_dequant_int4`	Matmul / GEMM / LoRA	TF	46.9	79.9	1.70×	—
13	`attention_forward_triton`	Attention	TF	55.6	93.0	1.67×	—
14	`bgmv_expand_slice`	Matmul / GEMM / LoRA	GB/s	212.1	353.3	1.67×	201.42×
15	`rope_transform`	Rotary / Position	GB/s	224.3	371.4	1.66×	—
16	`matmul_dequantize_int4`	Matmul / GEMM / LoRA	TF	37.6	59.7	1.59×	—
17	`batched_vecmat_mult`	Matmul / GEMM / LoRA	GB/s	74.5	115.5	1.55×	—
18	`layer_norm_triton`	Normalization	GB/s	5.0	7.5	1.50×	0.93×
19	`rope_backward_transform`	Rotary / Position	GB/s	124.3	185.5	1.49×	1.59×
20	`rotary_emb_nopad`	Rotary / Position	GB/s	169.6	226.4	1.33×	—

Modification-behaviour transition counts (§5). Table 9 reports the per-category transition counts across the 628 total iter-to-iter source diffs.

Table 6: Operator-family breakdown across Tier A. *Median iter5/iter1* is the within-cohort median peak-performance ratio. *High-gain* counts operators with iter5/iter1 $\geq 1.28\times$ (the band used for ablation set 1). *Near-parity* counts operators with iter5/iter1 $\in [0.95\times, 1.10\times)$ (the band where iteration produces only nuisance changes). Sorted by high-gain count.

family	# ops	median iter5/iter1	high-gain ($\geq 1.28\times$)	near-parity
Matmul / GEMM / LoRA	17	$1.21\times$	8	5
Quantize / Dequantize	5	$2.17\times$	5	0
Rotary / Position	7	$1.29\times$	5	1
Attention	10	$1.13\times$	3	5
Softmax / LogSumExp	11	$1.13\times$	2	5
Convolution	1	$9.71\times$	1	0
Layout	2	$2.57\times$	1	1
Scan / Cumulative	4	$1.14\times$	1	2
Reduction	4	$1.07\times$	1	2
Normalization	17	$1.00\times$	1	12
Elementwise	14	$1.00\times$	1	13
Other	3	$1.02\times$	0	2
Loss / Backward	6	$1.02\times$	0	6
Random / Stochastic	3	$1.00\times$	0	3
Activation	6	$1.00\times$	0	6
Embedding / Indexing / KV	8	$1.00\times$	0	8
Optimizer	1	$0.93\times$	0	0
All Tier A	119	$1.03\times$	29	71

Table 7: The 28 operators screened out of the main run, grouped by structural shortfall in `comp_instru`. Two operators marked † were eventually implemented under the Batch-26 baseline-reference amendment (§4.1); the remaining 26 were not attempted, plus one hardware-excluded op (`matmul_tma`, sm_90 TMA, not runnable on A40) totals 27 operators absent from the 157-op main corpus out of TritonBench-G's 184. Reasons summarised; full discussion in `result.md` §2.

Group	Operators (and shortfall)
Chunked linear attention / state-space / RWKV *Per-chunk recurrence structure and intermediate buffer layouts not derivable from natural language.*	`bmm_chunk_bwd`, `chunk_bwd_dqkg`, `chunk_delta_fwd`, `chunk_gate_recurrence`, `chunk_gated_attention`, `chunk_gla_fwd`, `chunk_gla_simple`, `chunk_linear_attn`, `chunk_retention`, `chunk_retention_ops`, `decay_cumsum`, `fused_recurrent_delta`, `fused_recurrent_hgrn`, `fused_recurrent_retention`, `fused_rwkv6_kernel`, `lightning_attention`
Matmul / scan scheduler variants *Distinguishing feature is the scheduler (persistent CTAs, stream-K, etc.), absent from the prompt.*	`matmul_persistent_triton`†, `streamk_matmul`, `iv_dependent_matmul`, `spinning_lock_reduction`
RBE / RMS-norm fused / miscellaneous *Unusual layouts, fused norm+matmul, or per-op kernels (5th-order spherical harmonics, nested-loops scan).*	`rbe_triton_transform`, `rms_matmul_rbe`, `rms_rbe_matmul`, `parallel_attention`, `parallel_retention_attention`, `diag_ssm_triton`†, `fifth_order_sph_harmonics`, `nested_loops_processing`

Table 8: The 6 operators with $> 10\times$ baseline speedup that we exclude from the headline "ex. artifacts" geomean in Table 1. These speedups are not clean algorithm-vs-algorithm wins — in every case the perf-driver supplies degenerate inputs (overlapping single-lora segments, per-element launches, scalar Python loops) that force the baseline to do redundant work our parallel kernel correctly elides.

op	reported	mechanism (degenerate input shape)
`sgmv_expand_slice`	1661.7×	Driver builds `size` overlapping single-lora segments; baseline redoes the whole matmul `size` times.
`bgmv_shrink_kernel`	262.8×	Single-lora `SPLIT_K` baseline; our N-block parallel kernel.
`bgmv_expand_slice`	201.4×	Same single-lora redundancy as above.
`sin_computation`	32.6×	Baseline launches per-element; ours is vectorised.
`reversed_cumsum_scalar`	13.5×	Baseline is a scalar Python loop vs. a Triton kernel.
`softmax_flaggems`	11.6×	Tiny softmax dominated by baseline launch overhead.

Table 9: Per-category transition counts across 157 ops $\times$ 4 transitions $= 628$ total. A single transition can fall into multiple categories. Categories are grouped by trajectory shape: early-and-fading (top), late-growing (middle), and convergence-marker (bottom).

Category	1→2	2→3	3→4	4→5	total
`AUTOTUNE_ADD`	96	23	3	11	133
`KEY_SHAPE_TUNE`	96	33	17	22	168
`BLOCK_SIZE_TUNE`	77	40	12	10	139
`CORRECTNESS_FIX`	33	14	2	2	51
`ALGORITHM`	24	6	1	1	32
`STRUCTURAL`	21	13	3	3	40
`FUSION`	13	8	5	2	28
`WARPS_STAGES_TUNE`	31	25	33	39	128
`AUTOTUNE_WIDEN`	1	8	5	16	30
`AUTOTUNE_REMOVE`	0	8	12	9	29
`TRIVIAL`	0	29	66	105	200

C Full per-operator ablation trajectories

Regression-prevention examples (§6). Table 10 lists the six Set 2 operators where the blind agent shipped a regression that the metric-fed agent prevented.

Table 10: Six set-2 operators where the blind agent shipped a regression that the metric-fed agent prevented.

op	unit	A1	A5	A5/A1	M5/M1
`cross_entropy1`	GB/s	558	373	0.67×	1.09×
`swiglu_backward`	GB/s	244	191	0.78×	1.08×
`swiglu_triton`	GB/s	508	432	0.85×	1.07×
`reversed_cumsum_scalar`	GB/s	46.6	34.3	0.74×	1.24×
`l2_norm_triton2`	GB/s	18.9	8.13	0.43×	1.06×
`matmul_triton_autotune`	GB/s	251	—	corr. lost	1.02×

Full Set 1 and Set 2 trajectories. Tables 11 and 12 report the full per-operator iter1–5 peak performance under both metric-fed (MK) and blind (AK) conditions, alongside the three within-/cross-run ratios M5/M1, A5/A1, and A5/M5. The unit per operator (GB/s or TFLOPS) is the same used in `ablation_set.md` and `ablation_set_2.md`.

D Additional materials

The 14-category $\times$ 157-row modification trace, the 28-op feasibility triage, and the 56-row baseline-comparison failure-cause table are provided in the supplementary at `claude_iter_changes.md`

Table 11: Full per-op trajectories, ablation Set 1 (30 high-headroom Tier-A operators). MK: main-experiment iter$K$ peak; AK: blind-ablation iter$K$ peak. Unit: GB/s for memory-bound, TF (TFLOPS) for compute-bound (matches `ablation_set.md`). $*$ flags `exe_acc` < 1.

#	op	unit	M1	M2	M3	M4	M5	A1	A2	A3	A4	A5	M5/M1	A5/A1	A5/M5
1	quantize_kv_copy	GB/s	69.9	76.2	248	275	251	70.0	101	70.0	70.2	70.0	3.59×	1.00×	0.28×
2	logsumexp_fwd	GB/s	177	627	628	630	630	177	304	342	305	341	3.57×	1.93×	0.54×
3	quantize_copy_kv	GB/s	73.6	77.4	250	285	249	73.9	113	73.0	72.6	72.9	3.38×	0.99×	0.29×
4	token_attn_reduceV	GB/s	95.8	276	170	276	276	95.4	181	317	181	317	2.88×	3.32×	1.15×
5	quantize_kv_transform	GB/s	70.6	77.7	269	270	153	71.8	70.8	70.7	70.1	71.2	2.17×	0.99×	0.46×
6	quantize_global	GB/s	192	359	361	355	353	192	191	193	192	193	1.84×	1.01×	0.55×
7	triton_matmul	TF	69.9	121	127	127	127	69.9	66.6	66.5	73.9	73.4	1.82×	1.05×	0.58×
8	matmul_triton2	TF	71.8	127	126	126	125	71.9	69.9	69.8	74.1	73.9	1.75×	1.03×	0.59×
9	matmul_dequant_int4	TF	46.9	80.1	80.0	79.8	79.9	46.5	58.3	59.2	46.4	46.4	1.70×	1.00×	0.58×
10	matmul_leakyrelu	TF	72.3	127	123	123	123	72.4	69.9	69.7	73.3	73.0	1.70×	1.01×	0.59×
11	triton_attention	TF	51.8	88.6	89.2	88.4	88.0	51.3	42.5	4.45	51.4	27.8	1.70×	0.54×	0.32×
12	attention_forward_triton	TF	55.6	93.7	92.9	93.2	93.0	50.2	43.5	49.9	56.8	56.8	1.67×	1.13×	0.61×
13	bgmv_expand_slice	GB/s	212	354	358	353	353	199	199	198	200	198	1.67×	1.00×	0.56×
14	rope_transform	GB/s	224	372	372	371	371	213	249	249	249	249	1.66×	1.17×	0.67×
15	dequantize_matmul	TF	63.4	105	104	105	104	62.3	61.0	59.3	62.7	62.9	1.65×	1.01×	0.60×
16	bmm_optimized	TF	74.0	117	120	118	118	75.9	73.6	73.0	75.7	75.3	1.60×	0.99×	0.64×
17	matmul_dequantize_int4	TF	37.6	59.8	59.9	59.8	59.7	36.9	35.3	35.5	36.8	28.8	1.59×	0.78×	0.48×
18	int_scaled_matmul	TF	91.2	141	141	142	142	88.7	107	107	86.4	139	1.55×	1.56×	0.98×
19	batched_vecmat_mult	GB/s	74.5	117	115	116	115	70.2	74.1	113	113	141	1.55×	2.01×	1.22×
20	triton_linear_activation	TF	35.3	53.8	53.7	53.8	53.4	36.1	35.9	36.2	35.9	36.1	1.51×	1.00×	0.68×
21	layer_norm_triton	GB/s	5.01	1.24	7.54	7.70	7.52	4.89	4.94	4.64	4.79	4.64	1.50×	0.95×	0.62×
22	rope_backward_transform	GB/s	124	185	185	185	185	118	124	124	124	124	1.49×	1.05×	0.67×
23	attention_fwd_triton2	TF	29.5	36.2	39.5	39.6	39.7	29.0	21.8	21.8	21.8	19.5	1.35×	0.67×	0.49×
24	rotary_emb_nopad	GB/s	170	233	228	229	226	158	171*	171*	171*	171*	1.33×	1.08×	0.76×
25	add_example	GB/s	423	558	570	555	557	423	423	423	428	429	1.32×	1.01×	0.77×
26	max_reduction	GB/s	4.55	5.69	6.48	5.95	5.95	4.53	4.53	4.61	4.53	4.63	1.31×	1.02×	0.78×
27	softmax_flaggems	GB/s	287	377	376	376	374	254	254	256	255	255	1.30×	1.01×	0.68×
28	dequantize_rowwise	GB/s	61.2	79.3	79.5	79.5	79.3	157	179	157	179	179	1.30×	1.14×	2.26×
29	rotary_transform	GB/s	508	652	653	653	654	488	526	529	524	530	1.29×	1.09×	0.81×
30	rotary_transform_ops	GB/s	509	652	653	653	653	483	524	529	526	530	1.28×	1.10×	0.81×

Table 12: Full per-op trajectories, ablation Set 2 (30 mild-headroom Tier-A operators excluding Set 1). Same column convention as Set 1. `matmul_triton_autotune` lost correctness blind at iter2–5 due to a generator bug specific to kernels with pre-existing autotune over block sizes (see §8).

#	op	unit	M1	M2	M3	M4	M5	A1	A2	A3	A4	A5	M5/M1	A5/A1	A5/M5
1	quant_transpose_kernel	GB/s	92.2	92.4	379	382	381	80.0	80.0	80.0	79.9	80.3	4.14×	1.00×	0.21×
2	flash_decode2_phi	GB/s	843	849	1064	844	1065	821	822	818	922	922	1.26×	1.12×	0.87×
3	reversed_cumsum_scalar	GB/s	47.8	59.1	56.7	58.5	59.3	46.6	42.2	34.7	46.7	34.3	1.24×	0.74×	0.58×
4	flash_decode2_llama	GB/s	899	1061	1059	1067	1070	870	934	861	939	932	1.19×	1.07×	0.87×
5	softmax_optimize	GB/s	547	631	644	644	645	486	475	483	475	478	1.18×	0.98×	0.74×
6	layernorm_fwd_triton	GB/s	551	647	647	647	647	552	556	554	556	553	1.17×	1.00×	0.85×
7	softmax_triton3	GB/s	556	644	645	645	645	557	562	561	561	561	1.16×	1.01×	0.87×
8	softmax_triton2	GB/s	558	645	645	645	645	556	562	558	561	561	1.16×	1.01×	0.87×
9	fast_layernorm	GB/s	550	634	635	634	635	550	545	537	540	537	1.15×	0.98×	0.85×
10	fast_rope_embedding	GB/s	569	648	651	651	650	543	591	592	591	592	1.14×	1.09×	0.91×
11	matmul_kernel	GB/s	224	258	256	259	256	225	175	186	247	248	1.14×	1.10×	0.97×
12	sgmv_expand_slice	GB/s	86.4	98.9	98.9	98.8	98.3	85.7	82.2	82.4	83.3	81.6	1.14×	0.95×	0.83×
13	triton_argmax	GB/s	521	522	600	594	589	521	516	418	521	521	1.13×	1.00×	0.88×
14	triton_softmax	GB/s	537	585	611	612	605	539	535	539	535	540	1.13×	1.00×	0.89×
15	cross_entropy1	GB/s	588	647	647	645	643	558	558	557	375	373	1.09×	0.67×	0.58×
16	swiglu_backward	GB/s	245	263	263	263	265	244	245	236	236	191	1.08×	0.78×	0.72×
17	swiglu_triton	GB/s	511	556	554	554	549	508	435	435	431	432	1.07×	0.85×	0.79×
18	rope_embedding	GB/s	608	647	648	648	649	565	593	594	594	594	1.07×	1.05×	0.91×
19	int8_matmul_quantization	GB/s	93.5	99.7	99.6	99.6	99.5	92.7	87.9	88.7	93.1	92.2	1.06×	0.99×	0.93×
20	l2_norm_triton2	GB/s	18.4	19.2	19.5	19.4	19.4	18.9	17.4	15.6	18.1	8.13	1.06×	0.43×	0.42×
21	layer_norm_liger	GB/s	615	649	649	650	649	579	582	567	567	567	1.06×	0.98×	0.87×
22	layer_norm_ops	GB/s	616	649	649	649	649	581	581	567	567	567	1.05×	0.98×	0.87×
23	chunk_cumsum_vector	GB/s	614	643	644	643	644	544	546	557	548	546	1.05×	1.00×	0.85×
24	destindex_copy	GB/s	698	690	721	727	728	697	725	725	725	725	1.04×	1.04×	1.00×
25	kldiv_compute	GB/s	232	232	236	232	240	569	570	578	577	585	1.03×	1.03×	2.44×
26	ksoftmax_triton	GB/s	632	653	652	652	652	593	592	575	574	574	1.03×	0.97×	0.88×
27	cosine_compute	GB/s	549	570	570	555	566	549	549	548	551	551	1.03×	1.00×	0.97×
28	attention_llama	GB/s	368	377	293	376	377	321	329	328	328	329	1.02×	1.02×	0.87×
29	square_matrix	GB/s	505	516	520	515	516	506	501	510	514	505	1.02×	1.00×	0.98×
30	matmul_triton_autotune	GB/s	250	256	256	261	255	251	—*	—*	—*	—*	1.02×	—	—

and `result.md`. The ablation kernel generator (`claude_gene/_ablation_gen.py`, `_ablation_gen2.py`), the parallel dispatcher (`claude_gene/dispatch.py`), and the heuristic edit classifier (`claude_gene/_iter_change_classify.py`) are released alongside.